# Лабораторная работа № 3

## Тема: «Обработка исключений. Модуль OS. Виртуальные окружения»

**Среда выполнения:** Google Colab + GitLab

**Прикладная задача:** «Пакетный импорт JSON-документов» — построить конвейер, который рекурсивно читает JSON-файлы из каталога, валидирует их по обязательным полям, изолирует ошибки по каждому файлу, пишет логи и формирует сводный отчёт.

### Часть I. Теория

### 1. Исключения

**Исключение (exception)** — объект, сигнализирующий об ошибке во время выполнения. Если его не обработать, программа завершится аварийно и покажет traceback.

Синтаксис:

```
try:
    # код, в котором может возникнуть ошибка
except SomeError as exc:
    # обработка
else:
    # выполняется, если ошибки не было
finally:
    # выполняется всегда
```

#### 1.1. Порядок выполнения

1. Интерпретатор входит в try и выполняет код.
2. Если исключения не было — управление уходит в else (если он есть), минуя except.
3. Если возникло исключение — ищется подходящий except. Первый совпавший выполняется, остальные игнорируются.
4. finally выполняется всегда: при ошибке, без ошибки, при return и даже при raise.

#### 1.2. Иерархия исключений

Все исключения наследуются от BaseException. Основные ветки:

| Класс | Назначение |
|---|---|
| SystemExit | sys.exit() |
| KeyboardInterrupt | Ctrl+C |
| GeneratorExit | закрытие генератора |

| Класс | Назначение |
|---|---|
| Exception | база для «обычных» ошибок |

Примеры внутри Exception:

| Ветка | Потомки | Пример |
|---|---|---|
| ArithmeticError | ZeroDivisionError, OverflowError | 1 / 0 |
| LookupError | IndexError, KeyError | [1, 2][5] |
| OSError | FileNotFoundError, PermissionError, IsADirectoryError | open("нет.txt") |
| ValueError | — | int("abc") |
| TypeError | — | "5" + 5 |
| AttributeError | — | (5).foo |

**Правило:** перехватывайте конкретные типы, а не except: и не except Exception: без нужды. Голый except: ловит в том числе KeyboardInterrupt.

### 1.3. Собственные исключения

```
class AppError(Exception):
    """Базовая ошибка приложения."""
class ValidationError(AppError):
    """Данные не прошли валидацию."""
```

Наследуйте от Exception. Имя класса заканчивается на Error. Сообщение должно объяснять, что случилось.

### 1.4. LBYL и EAFP

- **LBYL** (Look Before You Leap) — сначала проверить условие, потом действовать.
- **EAFP** (Easier to Ask Forgiveness than Permission) — действовать, а ошибку обрабатывать по факту.

В Python идиоматичен EAFP: между проверкой и действием состояние может измениться.

### 1.5. Лучшие практики

- Ловить конкретное исключение, а не всё подряд.
- Не использовать исключения для управления потоком.

- Всегда сохранять исходный traceback при оборачивании: raise NewError(...) from exc.
- Не проглатывать исключения молча (except: pass).
- Собственные исключения — под предметную область, а не «на всякий случай».

**2. Работа с файлами**

**2.1. Режимы открытия**

| Режим | Назначение | Создаёт? | Перезаписывает? |
|---|---|---|---|
| 'r' | Чтение | Нет | — |
| 'w' | Запись | Да | Да |
| 'a' | Дозапись | Да | Нет |
| 'x' | Создание | Да (ошибка, если есть) | — |
| 'r+' | Чтение и запись | Нет | Частично |
| 'b' | Бинарный режим | — | — |

**Правила:**

- Всегда указывать encoding="utf-8" для текстовых файлов. Иначе на Windows может подставиться cp1251, и кириллица превратится в кракозябры.
- Всегда использовать with open(...) as f: — файл закроется даже при исключении.
- Для больших файлов — построчное чтение: for line in f.

**2.2. JSON**

```
import json
with open("data.json", "w", encoding="utf-8") as f:
   json.dump(data, f, ensure_ascii=False, indent=2)
with open("data.json", "r", encoding="utf-8") as f:
   loaded = json.load(f)
```

- ensure_ascii=False — чтобы кириллица не превращалась в \uXXXX.
- json.loads / json.dumps — для строк; json.load / json.dump — для файлов.
- json.JSONDecodeError — исключение при битом JSON.

**2.3. CSV**

```
import csv
```

```
with open("data.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(rows)
```

newline="" обязательно, иначе на Windows появятся пустые строки.

**3. Модули os, os.path, pathlib**

**3.1. Модуль os**

| **Функция** | **Назначение** |
|---|---|
| os.getcwd() | Текущий каталог |
| os.chdir(path) | Перейти в каталог |
| os.listdir(path) | Содержимое каталога |
| os.mkdir / os.makedirs | Создать папку / цепочку папок |
| os.remove / os.rmdir | Удалить файл / пустую папку |
| os.rename(src, dst) | Переименовать/переместить |
| os.environ | Словарь переменных окружения |

**3.2. Модуль os.path**

| **Функция** | **Пример** |
|---|---|
| os.path.join(a, b) | "data/orders.txt" |
| os.path.basename(path) | "orders.txt" |
| os.path.dirname(path) | "data" |
| os.path.split(path) | ("data", "orders.txt") |
| os.path.splitext(path) | ("orders", ".txt") |

| Функция | Пример |
|---|---|
| os.path.exists / isfile / isdir | True / False |
| os.path.abspath(path) | /content/data/orders.txt |
| os.path.getsize(path) | размер в байтах |

### 3.3. pathlib — современный подход

```
from pathlib import Path
p = Path("data") / "orders.txt"
p.write_text("...", encoding="utf-8")
text = p.read_text(encoding="utf-8")
for child in Path("data").iterdir():
    ...
for f in Path("data").rglob("*.json"):
    ...
```

Полезные свойства: .name, .stem, .suffix, .parent, .exists(), .is_file(), .is_dir(), .mkdir(parents=True, exist_ok=True).

**Правило:** в новом коде — pathlib. os.path оставляем для унаследованного кода.

## 4. Модули и пакеты

**Модуль** — файл .py. **Пакет** — каталог с модулями и файлом __init__.py.

```
Варианты импорта:
import math                  # math.sqrt(4)
from math import sqrt        # sqrt(4)
from math import sqrt as sq  # sq(4)
from math import *           # плохо: засоряет пространство имён
```

**Идиома** if __name__ == "__main__": отделяет код для запуска от кода для импорта. При импорте модуля этот блок не выполняется.

__init__.py может быть пустым, а может реэкспортировать имена:

```
from .core import Engine
from . import utils
```

## 5. Виртуальные окружения, pip, requirements.txt

**Виртуальное окружение (**venv**)** — изолированная песочница для зависимостей проекта. Решает проблему: у проекта A нужна pandas==1.5, у проекта B — pandas==2.0.

```
python -m venv .venv          # создать
source .venv/bin/activate     # активировать (Linux/macOS)
```

```
.venv\Scripts\activate          # активировать (Windows)
pip install pandas              # установить
pip freeze > requirements.txt   # зафиксировать
```

requirements.txt — список зависимостей. Для сложных проектов — poetry, pip-tools с lock-файлом.

.env **и** .gitignore**:**

```
# .env
API_KEY=super-secret-key
DB_HOST=localhost
# .gitignore
.env
venv/
__pycache__/
```

Секреты — только в окружении, не в коде и не в Git.

**6. Логирование**

print не подходит для продакшена: нет уровней, меток времени, нельзя направить в файл.

**Уровни:** DEBUG < INFO < WARNING < ERROR < CRITICAL.

**Архитектура:**

- **Logger** — точка входа (logging.getLogger(__name__)).
- **Handler** — куда писать (консоль, файл, syslog).
- **Formatter** — как писать (формат строки).

**Ротация:** RotatingFileHandler, TimedRotatingFileHandler. Без неё лог вырастет в гигабайты.

**Ленивое форматирование:** logger.info("User %s", user), а не f-строка. Строка собирается только если сообщение пройдёт фильтр уровня.

**Никогда не логировать:** пароли, токены, персональные данные.

**Часть II. Задания**

**Задания на освоение концепции (1–10)**

**Задание 1. Функция vs процедура в обработке ошибок**

Напишите:

1. Процедуру log_error(path: str, reason: str) -> None, которая логирует ошибку через logging.warning с указанием файла и причины.
2. Функцию parse_int_safe(value: str) -> int | None, которая возвращает число или None при ValueError.

Проверьте обе на нескольких примерах. Объясните в комментарии, чем процедура отличается от функции.

**Задание 2. Все виды исключений**

Для каждого выражения предскажите тип исключения, затем проверьте в try/except:

```
1 / 0
int("abc")
[1, 2][5]
{"a": 1}["b"]
"5" + 5
open("no_such_file.txt")
```

Для каждого — покажите __mro__ и объясните, от кого наследуется.

**Задание 3. try / except / else / finally**

Реализуйте функцию safe_divide(a, b) -> float | None:

- в try — деление;
- в except ZeroDivisionError — лог и None;
- в else — сообщение об успехе;
- в finally — сообщение «операция завершена».

Продемонстрируйте: (10, 2), (10, 0), (10, "x"). Объясните, почему finally сработает даже при return внутри try.

**Задание 4. Своя иерархия исключений**

Создайте:

- IngestError(Exception) — база;
- ConfigError(IngestError) — настройки невалидны;
- DocumentError(IngestError) — проблема с одним документом, принимает path и reason.

Продемонстрируйте: каждое исключение поднимается и ловится. Покажите, что except IngestError ловит оба наследника.

**Задание 5. LBYL vs EAFP**

Для задачи «прочитать файл, если он существует»:

1. Реализуйте через LBYL: if os.path.exists(path): open(...).

2. Реализуйте через EAFP: try: open(...) except FileNotFoundError.

Объясните, в каком случае EAFP надёжнее.

**Задание 6. Файловые режимы**

Создайте файл test.txt в режиме 'w'. Запишите три строки. Прочитайте целиком, потом построчно. Откройте в режиме 'a', допишите строку. Откройте в режиме 'r' — убедитесь, что все 4 строки на месте. Объясните, что произойдёт, если открыть в режиме 'w' ещё раз.

**Задание 7. JSON: сериализация и десериализация**

1. Создайте словарь с русским текстом и списком.
2. Сохраните в файл через json.dump с ensure_ascii=False.
3. Прочитайте через json.load. Убедитесь, что данные совпадают.
4. Попробуйте загрузить битый JSON — поймайте json.JSONDecodeError.

**Задание 8. Работа с pathlib**

Создайте дерево каталогов:

text

data/

├── raw/

│ ├── a.json

│ └── b.json

└── processed/

Через pathlib.Path:

1. Создайте каталоги (mkdir(parents=True, exist_ok=True)).
2. Запишите в a.json и b.json любой текст.
3. Обойдите все .json через rglob.
4. Выведите для каждого: имя, размер, полный путь.

**Задание 9. Свой модуль**

Создайте файл jsonutils.py с функциями:

- read_json(path) -> dict — читает JSON, поднимает DocumentError при проблемах;
- write_json(data, path) -> None — пишет JSON с ensure_ascii=False.

В конце — блок if __name__ == "__main__": с демонстрацией.

Импортируйте модуль в ноутбук и используйте.

**Задание 10. Логирование с уровнями**

Настройте логгер:

- уровень INFO для консоли, DEBUG для файла app.log с ротацией;
- формат: время, уровень, имя логгера, сообщение.

Выведите сообщения всех пяти уровней. Объясните, почему DEBUG не появился в консоли. Проверьте содержимое app.log.

**Задания по прикладной задаче (11–20)**

**Прикладная задача:** «Пакетный импорт JSON-документов».

Сервис читает JSON-файлы из каталога inbox/. Каждый файл — карточка сущности с обязательными полями id и name. Файлы могут быть валидными, битыми, неполными, нечитаемыми. Один битый файл не должен ронять весь конвейер.

**Инициализация данных (одинакова для всех заданий 11–20)**

Запустите эту ячейку один раз — данные будут использоваться во всех заданиях.

```
import os
import json
import random
import shutil
from pathlib import Path
random.seed(42)
shutil.rmtree("inbox", ignore_errors=True)
Path("inbox/good").mkdir(parents=True)
Path("inbox/nested").mkdir(parents=True)
# Валидные файлы
Path("inbox/good/a.json").write_text(
    '{"id": 1, "name": "Alice", "age": 30}', encoding="utf-8"
)
Path("inbox/good/b.json").write_text(
    '{"id": 2, "name": "Bob", "age": 25}', encoding="utf-8"
)
Path("inbox/nested/c.json").write_text(
    '{"id": 3, "name": "Carol", "age": 28}', encoding="utf-8"
)
# Битые файлы
Path("inbox/broken.json").write_text(
    '{"id": 4, "name": }', encoding="utf-8"
)
Path("inbox/no_keys.json").write_text(
    '{"foo": "bar"}', encoding="utf-8"
)
Path("inbox/not_object.json").write_text(
    '[1, 2, 3]', encoding="utf-8"
)
print(f"Файл создан: inbox/")
print(f"Всего файлов: {len(list(Path('inbox').rglob('*.json')))}")
```

**Задание 11. Настройки из окружения**

Реализуйте класс Settings:

- source_dir: Path — каталог-источник (из os.environ["SOURCE_DIR"], по умолчанию inbox);
- output_path: Path — путь отчёта (из OUTPUT_PATH, по умолчанию catalog.json);
- required_keys: tuple[str, ...] — обязательные поля (из REQUIRED_KEYS, по умолчанию ("id", "name")).

Метод Settings.from_env() читает окружение, валидирует каталог и список полей. При ошибке — поднимает ConfigError.

**Задание 12. Обход каталога**

Реализуйте метод Ingester.iter_files() -> Iterator[Path], который рекурсивно обходит Settings.source_dir через Path.rglob("*.json") и возвращает отсортированный итератор.

Объясните, почему rglob предпочтительнее os.walk.

**Задание 13. Чтение одного документа**

Реализуйте Ingester.load_document(path: Path) -> dict:

1. Читает файл как текст (UTF-8).
2. Разбирает JSON.
3. Проверяет, что результат — словарь.
4. Проверяет наличие всех обязательных полей.

Каждая проблема — DocumentError с указанием файла и причины. Ошибки OSError и JSONDecodeError оборачивайте через raise ... from exc.

**Задание 14. Конвейер с изоляцией ошибок**

Реализуйте Ingester.run() -> dict:

1. Перебирает файлы из iter_files().
2. Для каждого вызывает load_document.
3. На DocumentError — logger.warning, добавляет в список ошибок, продолжает.
4. На любое другое исключение — logger.exception, тоже продолжает.
5. Возвращает отчёт:

```
{
 "source": "inbox",
 "total_files": 6,
 "imported": 3,
 "skipped": 3,
 "errors": [...],
 "records": [...]
}
```

**Задание 15. Сохранение отчёта**

Реализуйте Ingester._save(report) -> None:

- создаёт родительский каталог, если его нет (mkdir(parents=True, exist_ok=True));

- записывает JSON с ensure_ascii=False, indent=2;
- логирует факт сохранения.

При ошибке записи — поднимает ConfigError.

**Задание 16. Настройка логирования**

Реализуйте setup_logging() -> None:

- уровень INFO для консоли, DEBUG для файла ingest.log;
- RotatingFileHandler, 100 КБ, 3 бэкапа;
- формат: время, уровень, имя логгера, сообщение.

Замените все print в коде на соответствующие уровни логов.

**Задание 17. CLI с кодами выхода**

Реализуйте точку входа:

```
def main() -> int:
    try:
        settings = Settings.from_env()
    except ConfigError as exc:
        logger.error("Конфигурация невалидна: %s", exc)
        return 2
    try:
        report = Ingester(settings).run()
    except IngestError as exc:
        logger.error("Импорт прерван: %s", exc)
        return 1
    logger.info("Готово: обработано %d из %d",
                report["imported"], report["total_files"])
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```

Объясните, почему разные коды выхода — это важно для планировщика задач.

**Задание 18. Проверка сценариев**

Запустите конвейер на подготовленных данных. Убедитесь:

- 3 валидных файла попали в catalog.json;
- 3 битых пропущены с понятными сообщениями в логе;
- программа завершилась с кодом 0 (ошибки файлов не считаются провалом прогона).

Проверьте catalog.json и ingest.log.

**Задание 19. Секреты и .gitignore**

1. Создайте .env с настройками:

```
text
SOURCE_DIR=inbox
OUTPUT_PATH=catalog.json
REQUIRED_KEYS=id,name
```

2. Загрузите через python-dotenv.
3. Создайте .gitignore с содержимым:

```
.env
venv/
__pycache__/
*.pyc
*.log
.idea/
.vscode/
```

4. Убедитесь, что .env не попал в git status.

**Задание 20. Комплексный отчёт: единый pipeline**

Соберите всё в единый модуль:

1. Функция generate_report(input_dir: str, output_path: str, required_keys: tuple) -> dict:
   - создаёт Settings;
   - запускает Ingester;
   - возвращает отчёт.
2. Функция main() -> int — оркестратор: настраивает логирование, читает окружение, вызывает generate_report, печатает сводку и пути к созданным файлам.
3. Точка входа: if __name__ == "__main__": raise SystemExit(main()).
4. Все функции — с аннотациями и docstring.
5. Проверьте, что код проходит flake8 и mypy (если установлены).

**Критерии оценки:**

- Нет дублирования (DRY).
- Каждая функция делает одну вещь (SRP).
- Есть обработка ошибок.
- Соблюдён PEP 8.
- Используются pathlib, logging, venv / .env.

**Контрольные вопросы**

1. Чем else в try отличается от кода в конце блока try?
2. Почему finally выполнится, даже если внутри try был return?
3. Почему except: без типа — плохо? Что он может поймать лишнего?
4. Что такое LBYL и EAFP? Приведите пример, где EAFP выигрывает.
5. Зачем наследовать собственное исключение от Exception, а не от BaseException?

6. Что произойдёт, если открыть файл в режиме 'w' и он существует?
7. Зачем в open указывать encoding?
8. Чем pathlib.Path лучше os.path? Есть ли случаи, где os.path предпочтительнее?
9. Зачем нужен if __name__ == "__main__"?
10. Что такое __init__.py и почему без него пакет может работать не везде?
11. Зачем виртуальное окружение? Что произойдёт без него через полгода разработки?
12. Почему .env нельзя коммитить? Что делать, если он уже попал в репозиторий?
13. Зачем логирование, если есть print? Приведите три причины.
14. Что такое ротация логов и зачем она нужна?
15. Как logging.getLogger(__name__) помогает в больших проектах?

**Дополнительно**

1. Оформите выполненные задания из ноутбука в виде отдельного Jupyter-ноутбука с именем lab3_exceptions_files.ipynb. В ноутбуке должны быть:
    - титульный markdown-блок с указанием ФИО, группы и номера лабораторной работы;
    - отдельные ячейки с решениями заданий 1–20;
    - комментарии к коду (что делает функция, какие аргументы принимает, что возвращает).
2. Убедитесь, что ноутбук полностью выполняется сверху вниз без ошибок: последовательно запустите все ячейки через меню **Kernel → Restart & Run All**. При необходимости исправьте порядок ячеек и зависимости между ними.
3. Создайте в своём аккаунте на GitLab новый приватный репозиторий с именем python-labs (если он ещё не создан) и добавьте в него папку lab3.
4. Инициализируйте в папке lab3 локальный git-репозиторий, если его нет:

```
cd lab3
git init
```

5. Добавьте в папку lab3 следующие файлы:
    - lab3_exceptions_files.ipynb — итоговый ноутбук с решениями;
    - ingest.log — пример лога работы конвейера;
    - catalog.json — результат импорта;
    - README.md — краткое описание работы: цель, какие задания выполнены, какие функции реализованы, с какими ошибками столкнулись.
6. Создайте файл .gitignore, исключив из отслеживания служебные и сгенерированные файлы:

```
inbox/
.venv/
venv/
.env
.ipynb_checkpoints/
```

```
__pycache__/
*.pyc
*.log
.idea/
.vscode/
```

7. Зафиксируйте изменения и отправьте их в удалённый репозиторий:

```
git add .
git commit -m "ЛР3: исключения, файлы, логирование, окружение"
git push origin main
```

8. Убедитесь, что после git push файлы действительно появились в удалённом репозитории (откройте страницу репозитория в браузере).
9. В конце README.md добавьте раздел **«Ссылка на репозиторий»** и укажите полный URL вашего репозитория.
10. Ссылку на репозиторий и сам ноутбук прикрепите в качестве ответа на задание в системе дистанционного обучения.